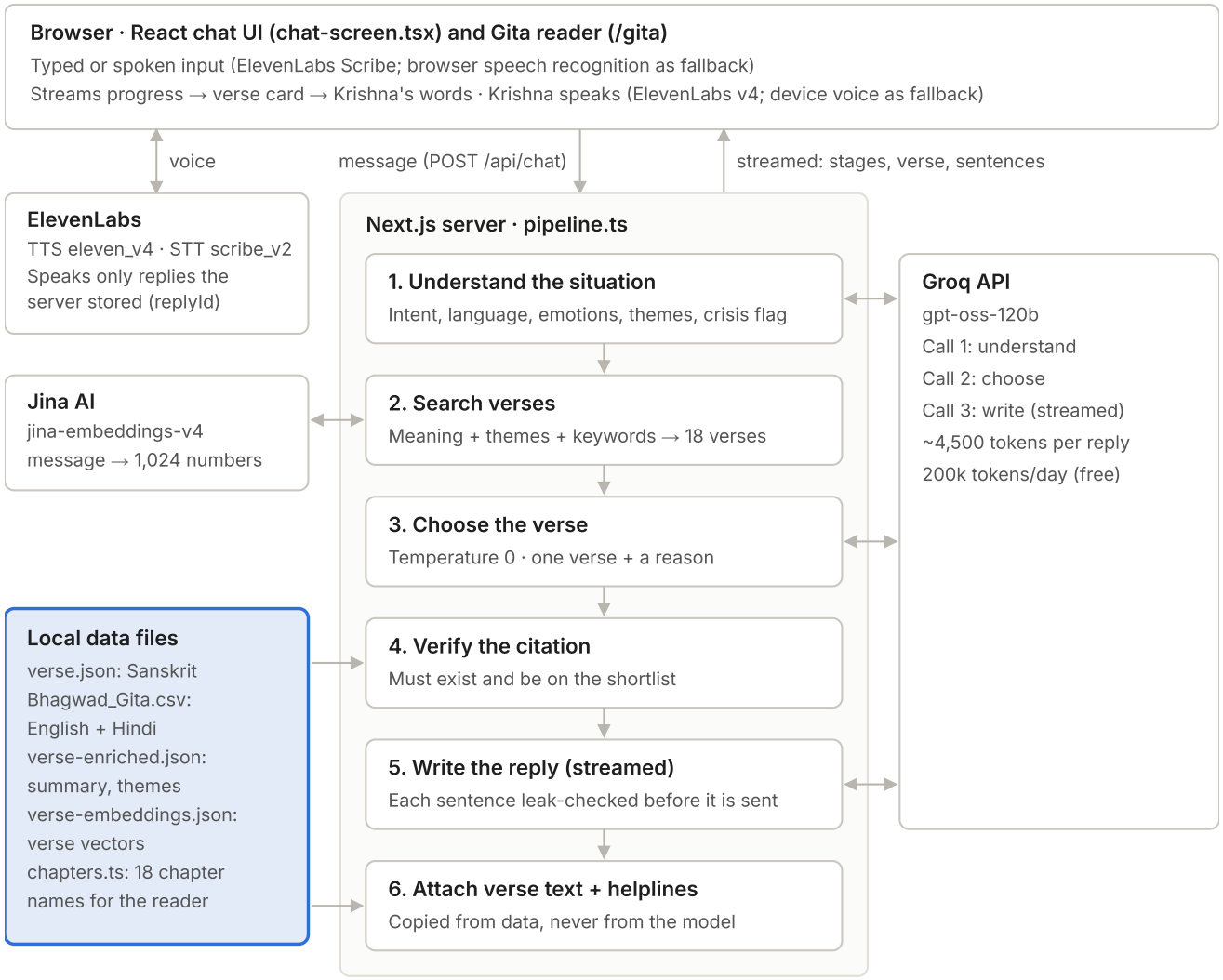
Speak with Krishna — How It's Built

Oct 7, 2026 · @Xvox Labs

A web app where a person shares a personal problem and "Krishna" answers with the Bhagavad Gita verse that best fits their situation, a plain summary of that verse, how it applies to them, and one practical step. People can type or talk, Krishna can answer aloud in a natural voice, and every chapter of the Gita can be read in the app. It is a Next.js + TypeScript app that combines retrieval over all 701 verses, an LLM on Groq, ElevenLabs voice, and validation so the app never shows an invented verse.

Architecture

Groq writes the words; every verse comes from local data



request flow · browser, 6 server steps, Groq, Jina, ElevenLabs, local data

Only the three Groq calls produce new words; the verse text, summary and helpline numbers always come from files shipped with the app. Jina is only asked to turn the message into a vector for search. ElevenLabs only speaks text the server already wrote and verified, and turns the user's recorded speech into text.

Tech stack

Everything is one Next.js app in TypeScript; there is no database and no separate backend service.

Layer	Technology	Version	Why it is used
Language	TypeScript	5.9	Type safety across UI, API and data scripts
Runtime	Node.js	18.20	Runs the Next.js server and the offline scripts
Web framework	Next.js (App Router)	15.5	Serves the chat page, the Gita reader pages and the `/api/chat`, `/api/tts`, `/api/stt` routes from one codebase
UI	React	19.3	Chat screen, verse cards, voice controls, Talk mode
Styling	Plain CSS with design tokens (`tokens.css`) + Google Fonts (Fraunces, Inter, Noto Serif Devanagari)	—	One consistent "night temple" look; correct Devanagari rendering for Sanskrit
LLM	Groq API, model `openai/gpt-oss-120b`	—	Understands the user's situation, chooses the verse, writes Krishna's reply; very low latency
LLM client	`openai` npm SDK (pointed at Groq's OpenAI-compatible endpoint)	5.23	One standard client with streaming; easy to switch provider later
Embeddings	Jina AI `jina-embeddings-v4` (REST API)	—	Semantic search: matches the meaning of a problem to verses, including Hindi/Tamil/Hinglish
Verse data	`verse.json` + `Bhagwad_Gita.csv`	701 verses	Sanskrit, transliteration, word meanings, English and Hindi translations
Voice out	ElevenLabs TTS `eleven_v4` (streamed MP3); browser `speechSynthesis` as fallback	—	A calm, natural Krishna voice that sounds the same on every device
Voice in	ElevenLabs Scribe `scribe_v2` + browser `MediaRecorder`; `SpeechRecognition` as fallback	—	Mic input and hands-free Talk mode in every browser, including Firefox

Layer	Technology	Version	Why it is used
Streaming	NDJSON over `fetch` (one JSON event per line)	—	Shows progress, the verse card and Krishna's words as soon as each is ready
Hosting	Vercel (Node runtime, 60 s limit)	—	Deploys on every push to `main`; keys kept in Vercel env vars
Tests	Node test runner via `tsx --test`	tsx 4.23	52 unit tests, no extra test framework
Scripts	`tsx`	4.23	Runs the TypeScript enrichment, embedding and evaluation scripts

How a reply is produced

Each message runs through a fixed pipeline in `src/lib/pipeline.ts`: three short LLM calls, one embedding call, and code checks between them. A full reply takes about 3–5 seconds; the verse card appears after about 3 seconds because the reply is streamed.

1. **Receive** — `POST /api/chat` validates the session id and message (max 2,000 characters) and loads the last 8 messages of that session. With `stream: true` the route answers with a stream of events instead of one JSON body.
2. **Crisis check** — a keyword check in English, Hindi, Hinglish and Tamil for suicide or self-harm (`safety.ts`). It runs again on the model's reading in step 3.
3. **Understand the situation (LLM call 1, temperature 0)** — the model turns the message into JSON: intent (problem, follow-up, greeting, about me, language request, unclear, off-topic, harmful), the reply language, emotions, one-line situation, what the person needs, 1–3 themes from a fixed list of 24, a crisis flag, and whether this continues the previous topic. Users can write in English or 9 Indian languages (Hindi, Tamil, Telugu, Kannada, Malayalam, Marathi, Bengali, Gujarati, Punjabi) or a mix; Krishna replies in their language.
4. **Short reply when no verse is needed** — greetings, "who are you?", unclear, off-topic and harmful messages get a brief conversational reply instead of a forced verse.
5. **Embed the query (Jina)** — message + situation + need is turned into a 1,024-number vector with task `retrieval.query`.
6. **Retrieve 18 candidate verses** (`retrieve.ts`) — score = semantic similarity to each verse's stored vector + a small bonus for matching themes + a small keyword bonus. Pure narrative verses (lists of warriors, conches) are excluded; Arjuna's own laments

are down-ranked. If Jina is unavailable the same function falls back to theme + keyword scoring.

7. **Choose the verse (LLM call 2, temperature 0)** — the model sees the situation and the 18 candidates (meaning, themes, situations it helps with, situations it is *not* for, translation) and returns one verse (rarely two) with a reason. Temperature 0 makes the same problem get the same verse.
8. **Verify** (`verify.ts`) — the cited verse must exist in the corpus *and* be on the shortlist; anything else is dropped and the model is asked once more, or the app honestly says no verse fits. If a new topic would repeat an earlier verse, it asks once for a different one. The verified verse card is sent to the browser right away.
9. **Write Krishna's reply (LLM call 3, temperature 0.6, streamed)** — JSON with three parts: acknowledge the situation, connect the verse to it, one practical step. As the model writes, the server releases the text only in whole sentences, and only after each sentence passes the leak checks (`stream-reply.ts`). Replies containing verse numbers or Devanagari are rejected and rewritten once; otherwise a safe fixed sentence is used.
10. **Respond** — the server attaches the verse's Sanskrit, translation, Hindi and summary **from the data files**, never from the model, plus a helpline card if the crisis check fired. It saves the turn to the session with a `replyId` and the exact text the voice may speak, and sends the final reply, which replaces what was streamed.

Data

All verse data is in files in the repo and loaded into memory once per server process; no database is needed for 701 verses.

File	Contents	How it was made
`verse.json`	701 verses: Sanskrit, transliteration, word meanings	Source data, never edited; 4 verses with mismatched Latin fields are hidden in code
`Bhagwad_Gita.csv`	English and Hindi translation per verse, chapter names	Source data
`verse-enriched.json`	Per verse: plain-English summary, 1–3 themes, 3–5 real-life situations it helps with, situations it is *not* for, teaching vs narrative flag	Generated once by `npm run enrich` (Groq, ~60 calls, resumable), then spot-checked

File	Contents	How it was made
`verse-embeddings.json`	One 1,024-number vector per verse (~3.8 MB, base64)	Generated once by `npm run embed` (Jina, `retrieval.passage`), not yet run
`src/lib/chapters.ts`	18 chapters: English meaning and a one-line summary of each	Written by hand for the Gita reader pages

Why the enrichment step matters: before it, every verse in a chapter shared one chapter-level theme and had no summary, so search could only match the user's exact words against translations. People describe problems ("my dad died, I can't focus at work") in words that rarely appear in 2,000-year-old verses. The summaries and situations are written in modern language, so both the search and the verse-choosing step can match on meaning. The summary is also what the user sees under each verse.

Accuracy and safety safeguards

The main risk in a scripture app is an LLM inventing or misquoting a verse, so the model never supplies scripture text.

- **No invented verses** — the model only returns chapter and verse numbers; the server looks them up and attaches the real text. A number not in the corpus or not on the shortlist is rejected.
- **Right verse for the feeling** — each verse carries "not for" notes (e.g. the famous verse on detachment from results is not for someone grieving), and the selection prompt matches the feeling, not just the topic.
- **Consistent answers** — understanding and selection run at temperature 0.
- **Honest when nothing fits** — the model may answer "no strong match"; the app then says so instead of forcing a verse.
- **Clean reply text, even while streaming** — verse numbers are always blocked; Devanagari is blocked in replies not written in it; in Hindi or Marathi, quoting three or more words of the verse is blocked. Streamed text is checked sentence by sentence before the user sees it, so scripture only appears in the verified verse card.
- **Crisis handling** — suicide or self-harm signals show a helpline card (Tele-MANAS 14416, emergency 112) above the reply, the reply is told to urge reaching out, the voice reads the crisis message first, and Talk mode pauses after speaking.
- **Prompt injection** — the user's text is always passed to the model as quoted data, never as instructions; harmful requests are declined calmly.

- **Voice credits cannot be abused** — the speech route only speaks a reply the server stored (by `replyId`), never arbitrary text; per-IP limits of 30 speech and 20 transcription requests per 10 minutes; uploads capped at 2 MB and 60 seconds.
- **Graceful failure** — if Jina fails, search falls back to keywords; if Groq fails or hits its limit, the user gets "please try again" instead of an error page; if ElevenLabs fails, the device voice and browser speech recognition take over.
- **Secrets** — API keys live only in `.env.local` (git-ignored) and Vercel env vars; error logs redact Groq, OpenAI, Google, Jina and ElevenLabs key patterns.

Voice

Voice now runs through ElevenLabs behind our own server routes, so Krishna sounds the same on every device and the mic works in every browser. The API key never reaches the browser. If ElevenLabs is not configured, out of credits or down, the app falls back to the browser's built-in voice and speech recognition automatically.

- **Listen** — each reply has a Listen button; an optional setting speaks every new reply. The browser plays `/api/tts`, which streams ElevenLabs MP3 (`eleven_v4`, a calm male voice at speed 0.95) so audio starts in about a second. Krishna reads the acknowledgement, the connection and the step, not the Sanskrit. Replies are cached in the browser for an hour, so pressing Listen again costs no credits.
- **Speak** — a mic button records with `MediaRecorder` and shows a level meter; the audio is transcribed by ElevenLabs Scribe (`scribe_v2`) and placed in the box for review before sending. Mic language: English (India), Hindi or Tamil.
- **Talk to Krishna** — a hands-free loop: listen → auto-send after about 1.2 s of silence → Krishna speaks → listen again. It is a small state machine (`nextTalkState` in `src/lib/speech.ts`) with states off, listening, thinking, speaking and paused, plus a voice-activity detector (`nextVadState`). Interrupt is by tap or Space, not by voice: the mic is never open while Krishna speaks, so it cannot hear his own voice. Two silent turns pause the loop.
- **Browser support** — speaking and voice input now work in Chrome, Edge, Safari and Firefox. The settings panel tells the user that their voice and Krishna's replies are processed by ElevenLabs.

Reading and devotee features

- **Read the Gita** — `/gita` lists all 18 chapters; `/gita/1` to `/gita/18` show every verse with Sanskrit, meaning, and expandable translation, transliteration, word meanings and

Hindi meaning. Pages are pre-rendered at build time; links like `/gita/2#v47` open and highlight one verse.

- **Verse of the Day** — one teaching verse per day, changing at midnight IST, shown on the home page.
- **My Gita** — save any verse; saved verses stay on the device (no account needed).
- **Share as image** — any verse becomes a 1080×1350 image card for WhatsApp or Instagram.

Testing and the accuracy benchmark

There are 52 unit tests (`npm test`, all passing) and a golden-set benchmark for verse accuracy (`npm run eval`).

- **Unit tests** cover: corpus loading and integrity (701 verses, SHA-256 of the source, every verse enriched); retrieval ranking with fixture vectors and the keyword fallback; verse verification (unknown or off-shortlist verses rejected, max two); leak checks in several languages; crisis detection in four languages; the voice helpers, voice-activity detection and Talk-mode state machine; the ElevenLabs client (with a stubbed network); rate limits; Verse of the Day; and the streaming sentence gate.
- **Golden set** — `src/lib/eval/golden-set.ts` has 58 realistic problems (grief, exam fear, anger, guilt, loneliness, addiction, career choice, faith and more, including Hinglish and Tanglish), each with 3–17 acceptable verses.
- **What it measures** — recall@5 and recall@18 (is a good verse in the shortlist?) and top-1 accuracy (did Krishna choose a good verse?). Flags compare modes: `--keyword` vs semantic, `--raw` to skip the situation step.
- **First baseline** — keyword search only, partial run before the Groq daily limit: 10 of the first 16 cases chose an acceptable verse, including all 5 grief cases and the Hinglish/Tanglish ones. Misses were mostly restless-mind and self-worth cases, which semantic search targets. The acceptable lists are one person's judgement and should be reviewed by someone who knows the Gita.

Current status, limits and next steps

The pipeline, enrichment, ElevenLabs voice, streaming, the Gita reader and tests are done; semantic search is built but switched off until the verse embeddings are generated.

Item	Status / limit	Next step
Semantic search	Code ready; `verse-embeddings.json` not generated, so keyword fallback is active	Run `npm run embed`, then `npm run eval` to compare with the keyword baseline
Groq free tier	200,000 tokens/day; ~4,500 tokens per reply ≈ 40 replies/day	Upgrade to Groq Dev tier before real users, or move the situation step to a smaller model
Jina v4 licence	Free, but licensed for non-commercial research only	Switch `JINA_EMBED_MODEL` to `jina-embeddings-v5-text-small` and re-run `npm run embed` before any commercial launch
ElevenLabs cost	Creator plan, ~150K credits/month; a reply is ~600–900 characters; the `eleven_v4` discount ends Oct 12, 2026	Set `ELEVENLABS_TTS_MODEL` to a cheaper model (e.g. `eleven_flash_v2_5`) if needed, no code change; set a usage cap in ElevenLabs
Sessions and rate limits	Kept in server memory (last 8 messages); lost on restart, not shared across servers, so on Vercel a follow-up or the Listen button can hit an instance without that reply	Move both to Redis (e.g. Upstash) before real traffic
Vercel settings	The ElevenLabs variables must be set in Vercel as well as `.env.local`	Add `ELEVENLABS_API_KEY`, `ELEVENLABS_VOICE_ID`, `ELEVENLABS_TTS_MODEL`, `ELEVENLABS_STT_MODEL` and redeploy
Accounts and privacy	No login; conversations are not stored beyond memory; voice audio and reply text are sent to ElevenLabs	Decide on auth, data retention and a privacy notice before launch
Speaking while streaming	Voice starts once the full reply has arrived	Optional: speak sentence by sentence as they stream
Benchmark labels	Acceptable verses chosen by one person	Have a Gita teacher review `golden-set.ts`

Item	Status / limit	Next step
Secrets hygiene	Fixed: `.env.example` was removed from the repo and git history holds no real keys; keys live only in `.env.local` and Vercel	Rotate the Jina key if it was ever shared outside the team